



Pourquoi une base de données pour PESSORA ?

Sécurité, évolutivité et autonomie du site

DOSSIER TECHNIQUE & STRATÉGIQUE



AUJOURD'HUI, TOUT EST « EN DUR »

Situation technique actuelle

- **Données statiques** : Menu, produits, événements et infos du bar sont définis dans des fichiers TypeScript.
 `src/data/menuData.ts, src/data/infoData.ts`
- **Processus rigide** : Événements, partenariats et horaires suivent la même logique.
- **Mises à jour complexes** : Changer un prix ou une boisson nécessite de modifier le code source, puis de redéployer tout le site.
- **Risques opérationnels** : Dépendance totale au développeur et risque d'erreur lors des déploiements pour des changements mineurs.



```
src/data/menuData.ts

export const
menuData
= [
  {
    id: "latte",
    name: "Matcha Latte",
    price: 5.50,
    // Changer ici nécessite
    // un redéploiement !
  },
];
```

 Code source verrouillé



Constat Critique

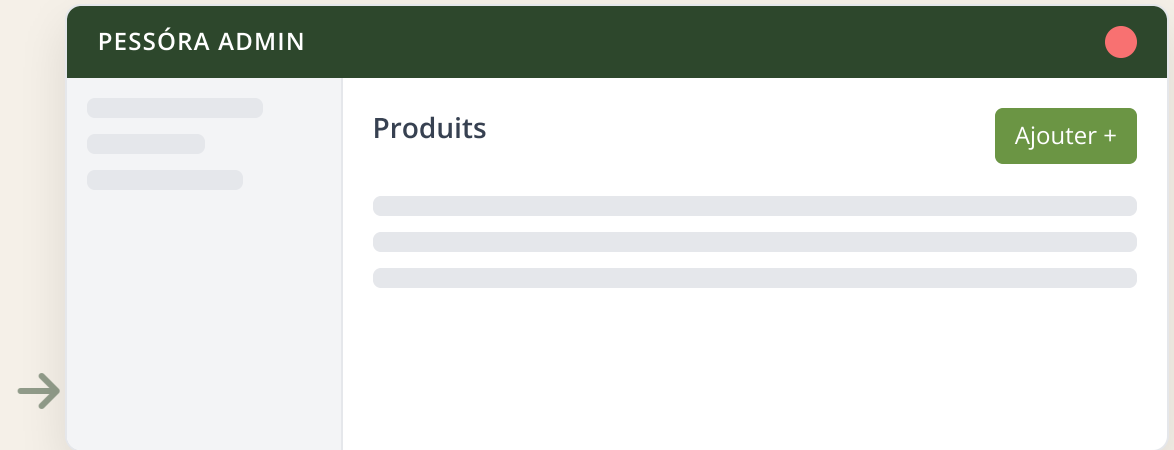
Le site ne peut pas évoluer de manière autonome. Chaque modification de contenu, aussi minime soit-elle, impose de toucher à la structure même du code.

L'ADMIN EST PRÊT

Mais sans base de données, il tourne à vide



- **Interface existante** : L'espace administrateur est déjà développé pour gérer les produits, événements et l'upload d'images.
- **Architecture Supabase** : Tout est conçu pour fonctionner avec les tables `products`, `events` et le `storage`.
- **Problème actuel** : Le client Supabase est un "mock" (`src/lib/supabaseClient.ts`). Les ajouts ne sont pas sauvegardés et le site continue d'afficher les fichiers statiques.
- **Objectif** : Connecter une vraie base pour permettre à l'équipe PessÓra de mettre à jour le site instantanément.



L'interface est une coquille vide tant qu'elle n'est pas reliée à la base de données réelle.



Le Bénéfice Immédiat

La BDD active l'admin : l'équipe devient autonome et le site vit en temps réel sans développeur.

VRAIS COMPTES ET FIDÉLITÉ



De l'utilisateur "invité" au membre engagé



Limitation actuelle (Mock) :

Connexion factice (localStorage). Si l'utilisateur change de téléphone ou vide son cache, il perd tout.

`src/contexts/AuthContext.tsx`



Authentification Réelle (Supabase) :

Comptes sécurisés, profils persistants et synchronisation multi-appareils.



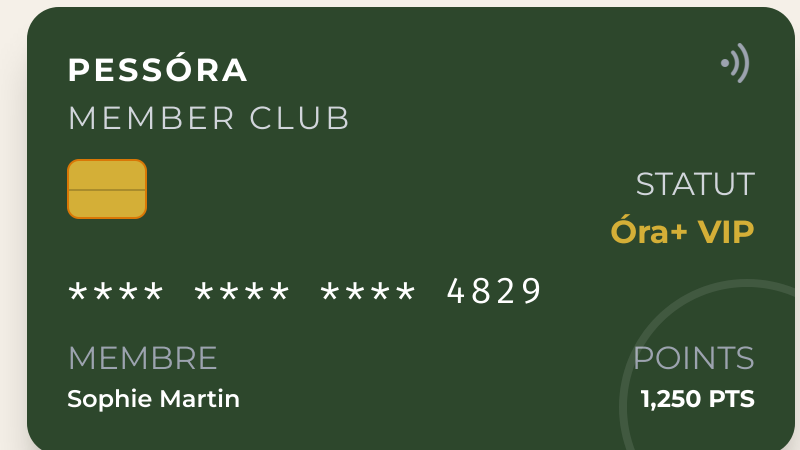
Programme Fidélité (Óra+) :

Stockage centralisé des points, historique des achats et gestion des niveaux (Starter, Premium, VIP).



Sécurité des Données :

Protection des données sensibles (RGPD) via Row Level Security (RLS).



Données stockées en BDD :

table: profiles, loyalty_points



Impact Business

Pas de "vrai" club membre ni de programme de fidélité possible sans une base de données. La BDD transforme des visiteurs anonymes en clients identifiés et fidélisés.

STRIPE ET APPLE WALLET

La limite du modèle actuel

- **Rôle de Stripe** : Il gère parfaitement les paiements et le cycle de vie de l'abonnement (actif, impayé, annulé).
- **Projet Apple Wallet** : Les cartes membres seront bientôt déployées via un prestataire externe pour être scannées en boutique.
- **Le défi technique** : Le prestataire Wallet a besoin de données précises (Nom, Niveau, Points) pour générer la carte.
- **Limite critique** : Stripe ne porte pas l'identité membre, le niveau de fidélité, ni le lien technique avec la carte Wallet.



Le constat

Stripe seul ne suffit pas. Il gère l'argent, mais pas l'identité du membre nécessaire pour faire fonctionner les cartes Apple Wallet et la fidélité.

POURQUOI STRIPE NE SUFFIT PAS



Le paiement est géré, mais l'identité membre reste à construire



Ce que Stripe gère

L'aspect transactionnel

- ✓ Paiement récurrent & statut (actif/impayé)
- ✓ Factures et reçus automatiques
- ✓ Gestion des moyens de paiement
- ✓ Webhooks (événements techniques)

Stripe = "Le client X a payé 9€"



Ce dont PessÓra a besoin

L'expérience Membre (Non couvert par Stripe)

- Identité Membre Unifiée (Profil, préférences, avatar)
- Niveaux & Avantages (Free / Starter / VIP)
- Points de Fidélité (Óra+) (Cumul & historique)
- Lien Apple Wallet 🍏 (Carte ↔ Membre)
- Cohérence Multi-canal (Site, App, Kiosque)

Besoin d'une BDD pour centraliser tout ça



Conclusion Technique

Il faut une **BDD PessÓra centrale** qui reçoit les infos de paiement Stripe mais qui reste maître de l'identité membre, des points et du lien avec les cartes Wallet.

BDD PESSÓRA : LE MAILLON MANQUANT



L'architecture technique pour relier Stripe et Apple Wallet

Source de Vérité "Membre"

La BDD stocke le profil unique, le niveau (Tier), les points fidélité et les identifiants techniques (IDs Stripe & Wallet).

Intégration Stripe (Paiements)

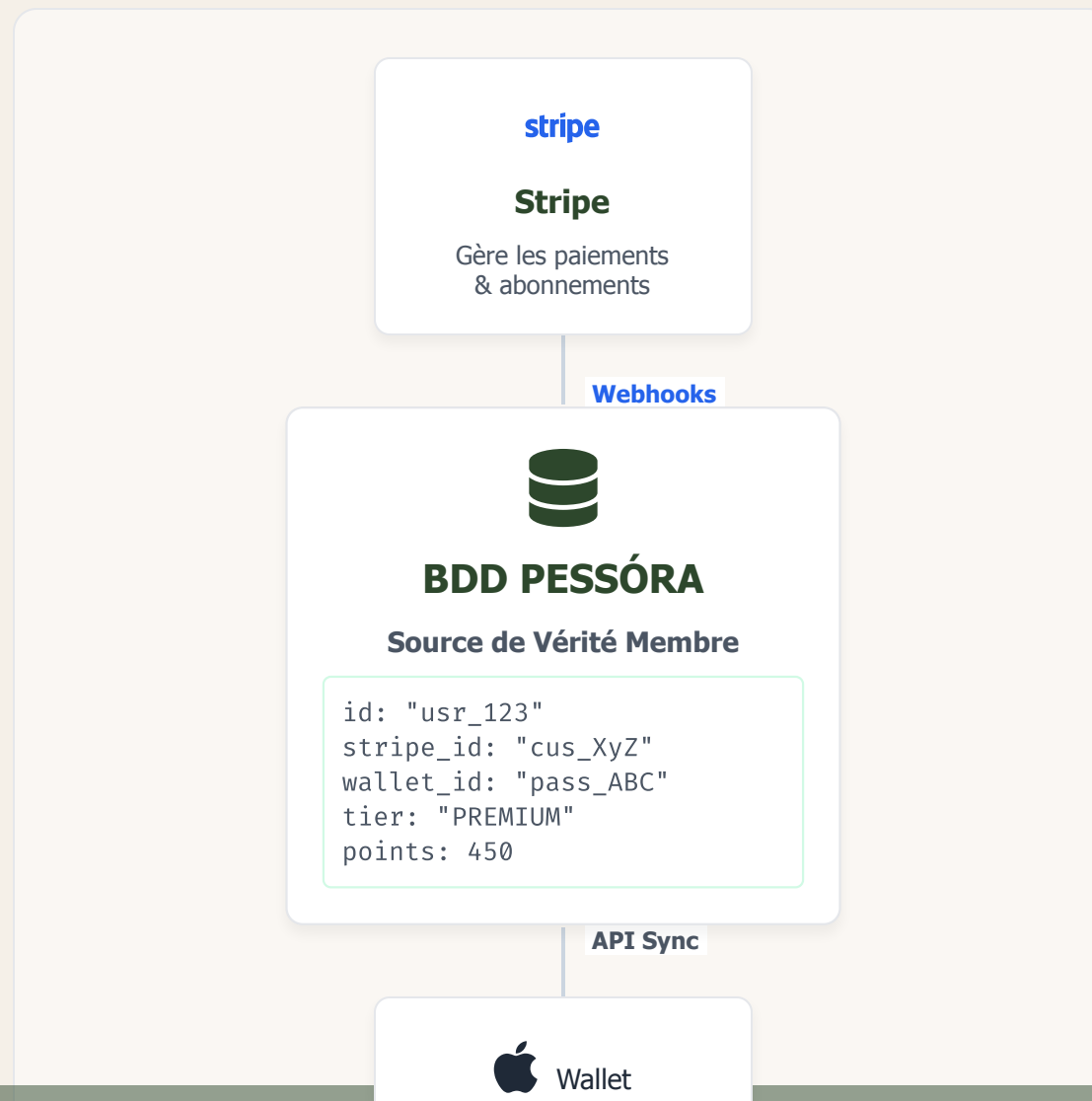
 **Webhooks** : Réception automatique des événements (paiement réussi, abo annulé).

 **Mapping** : Association automatique stripe_customer_id ↔ member_id.

Intégration Apple Wallet

 **API Prestataire** : La BDD pilote l'émission et la mise à jour des cartes.

 **En Boutique** : Scan → Lookup BDD → Application des avantages.

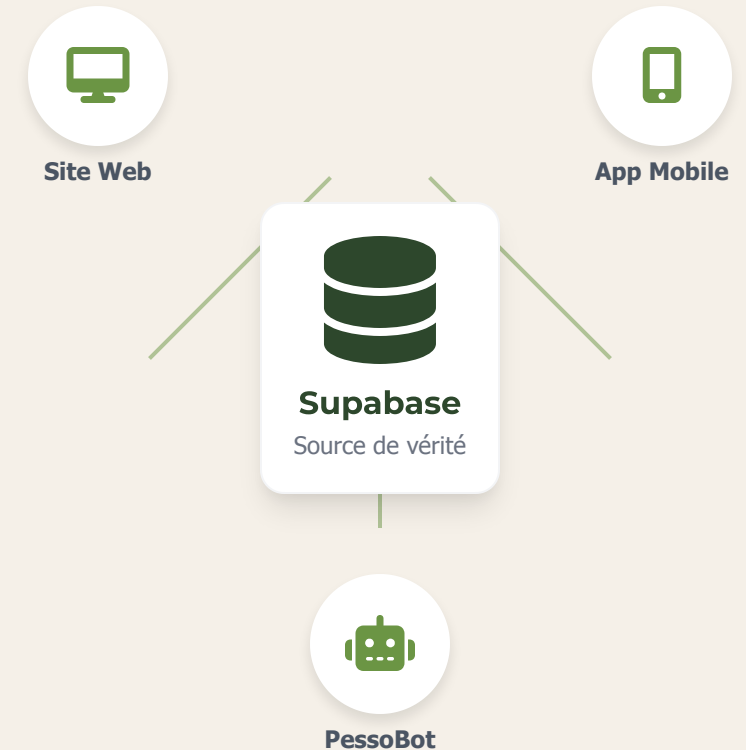


UNE SOURCE DE VÉRITÉ UNIQUE

Centralisation et cohérence cross-canal



- **Menu unifié** : Le menu affiché sur le site provient directement de l'admin (table products). Fini les discordances.
- **Événements synchronisés** : La page « Événements » lit la table events. Une mise à jour admin se reflète instantanément.
- **Infos pratiques centralisées** : Horaires, adresse et contacts sont stockés en base, évitant l'éparpillement.
- **Cohérence multi-plateforme** : La même base alimentera le futur bot, l'application mobile et les bornes en boutique.



Avantage Stratégique

La base de données élimine les risques de doublons et de désynchronisation. Elle garantit que l'information est fiable et identique partout, en temps réel.

CE QUE LA BDD PERMET POUR LA SUITE



Nouvelles opportunités & Évolutions métier



Réservations & Commandes

Capacité de stocker et gérer les créneaux horaires, suivre les statuts de commande (reçu, en préparation, prêt) et conserver l'historique des transactions.



Analytics & Insights

Analyse précise des produits les plus vus et achetés, identification des pics de fréquentation. Des données dérivées des tables pour piloter l'activité.



IA Pessobot Personnalisée

Un assistant intelligent capable de recommander des produits basés sur l'historique et les préférences du membre, grâce au compte utilisateur unifié.



Expérience Multi-canal

Une architecture centralisée servant de socle commun pour le site web, la future application mobile et potentiellement une borne kiosque en boutique.



Vision Stratégique

La base de données n'est pas qu'un outil de stockage : c'est le **socle technique indispensable** pour activer toutes les futures fonctionnalités métier.

DONNÉES PERSONNELLES ET SÉCURITÉ

Protection, RGPD et Conformité



Conformité RGPD

Gestion stricte des données personnelles (emails, identités, historiques) dans un cadre légal.



Contrôle d'Accès (RLS)

Row Level Security (Supabase) garantit mathématiquement que chaque utilisateur ne voit que ses propres données.



Sécurité du Code

Aucun secret API sensible n'est exposé dans le code client. La logique critique reste côté serveur.



Audit & Standards

Architecture validée selon les règles d'audit (réf. Karibloom) pour une traçabilité totale.



Confiance & Sérénité

Une base de données bien conçue est le socle indispensable pour garantir la **confidentialité** des membres et la **conformité** légale du projet.

FLUX SÉCURISÉ

 Client / Site Web
Authentification (JWT)



 **Supabase Auth**
Vérification d'identité



 Base de Données
Politiques RLS : "Select if uid = user_id"

 Données isolées & protégées

EN RÉSUMÉ

Pourquoi passer à une base de données ?



🎯 Points Clés

- ✓ **Autonomie Totale** : L'équipe gère le menu et les événements sans développeur.
- ✓ **Club Membre** : Gestion de vrais comptes, abonnements et historique.
- ✓ **Fidélité Óra+** : Programme de points et avantages personnalisés.
- ✓ **Scalabilité** : Prêt pour les commandes et le multi-canal.
- ✓ **Sécurité** : Données RGPD protégées et isolées.

Enjeu	Sans BDD (Actuel)	Avec BDD (Cible)
Mise à jour Menu	</> Déploiement code	✍ Admin instantané
Comptes Membres	🚫 Mock (Faux comptes)	👤 Auth + Profils réels
Fidélité (Óra+)	Impossible à gérer	★ Points & Historique
Cohérence Données	Risque de doublons	🗄 Source unique
Commandes	Non envisageable	🛒 Tables dédiées
Données Perso.	LocalStorage (Peu sûr)	🔒 Sécurisé (RLS)

FEUILLE DE ROUTE TECHNIQUE

1 **Configuration Supabase**
Création tables, Auth, Politiques RLS



2 **Connexion Frontend**
Remplacement des mocks par le client réel



3 **Migration & Lancement**
Import menu existant et déploiement